
wg-client Documentation

Release 8.0.4

Gene C

Oct 04, 2026

WIREGUARD CLIENT TOOL MANUAL:

1	wg-client	1
1.1	Overview	1
1.2	Why I made wg-client	1
1.3	Key features	2
2	Getting Started	3
2.1	DNS Notes	3
2.2	wg-client application	4
2.3	wg-client-gui application	4
2.4	Sudoers	4
3	Recent Changes	7
4	Configuration	9
5	Options	11
5.1	wg-client	11
6	Log files	13
7	Appendix	15
7.1	Migration from Pre 8.x	15
7.2	Installation	15
7.3	Dependencies	16
7.4	Log files	16
7.5	License	16
8	Man Pages	17
8.1	resolv-manager	17

WG-CLIENT

Migration required from pre 8.x versions

If you are updating from pre 8.x versions please see the Appendix to user newer versions *Migration from Pre 8.x*.

1.1 Overview

wg-client makes it simple for any user to start and stop a wireguard client on Linux. The command line program, *wg-client*, provides all the functionality. A GUI tool makes it super convenient. The GUI tool runs the command line client and thus offerses exactly the same functionality in a user friendly application.

A companion package, *wg-tool* is for adminstering the server side of things.

wg-client also has an option to invoke ssh to create a remote listening port connected back to local (client end) ssh daemon.

This can be useful to facilitate remote ssh back to client computer if it's needed. For example; it can be used to provide access to a git repo on the client, or for remote backups of laptop, or even for admin to login to client should the need arise.

All git tags are signed with arch@sapience.com key which is available via WKD or download from <https://www.sapience.com/tech>. Add the key to your package builder gpg keyring. The key is included in the Arch package and the source= line with *?signed* at the end can be used to verify the git tag. You can also manually verify the signature

1.1.1 Documentation

The manual provides detailed information about using wg-client. It is available in HTML and PDF formats installed under */usr/share/wg-client/docs*

It is also available at: [readthedocs](#).

1.2 Why I made wg-client

After building *wg-tool* which simplified administraion of wireguard servers, I needed a simple way for non-tech (unprivileged) users to connect their laptops to the server.

Thus wg-client was born. The gui client makes it really simple for non-tech users, though I find it pretty convenient as well.

1.3 Key features

- Graphical app makes it simple for any user to get VPN running.
- Standalone tool makes it easy to test and also keeps sudo outside of gui to minimize any security implications. The gui relies completely on the command line tool to do the real work.
- resolv-manager is written on C. It uses capabilities and is designed to be run by non-root users. When run as root, it drops privileges to (user, group) = (*nobody*, *nobody*).. Code has been checked and is clean using: - clang-tidy - valgrind - compiler based sanitizers
- python code is checked with mypy

GETTING STARTED

2.1 DNS Notes

When using any VPN client, it is standard practice to ensure all DNS traffic is routed through the VPN tunnel in order to trusted DNS servers. While this isn't a hard requirement, and is not strictly applicable with split routing, it is always desirable from a security standpoint.

There is a little gotcha to be aware of. The system may, at times, modify */etc/resolv.conf* removing the safety of the VPN provided DNS servers to different ones. This also breaks LAN related services.

Consequently, it is important to be identify if this happens and deal with it. It is handled is by saving any newly resolv.conf file and then restoring the correct one that should be used with wireguard. The resolv.conf that was just saved is the the right file to use when the VPN is turned off but not while it is runninf. By saving a copy, it can be restored when the wireguard VPN shuts down.

What causes this to happen? It can happen for different reasons. It can happen if the WiFi network changes, after a sleep resume cycle or when a DHCP lease is renewed.

While wireguard itself uses UDP and is mostly indifferent to such network changes, nonetheless it is quite important to keep DNS properly managed.

Taking care of all of this is eactly what *resolv-manager* does.

To ensure that DNS resolver files are all managed correctly requires a copy of the standard resolv.conf file saved to */etc/resolv.conf.saved* and the wireguard version to */etc/resolv.conf.wg*.

With those in place *resolv-manager* will keep track of any changes. It will save any updated standard */etc/rsolv.conf* file and put back the appropriate one for wireguard.

After creating */etc/wg-client/wireguard-resolv.conf* with the the DNS nameservers provided by the wireguard server end, then set the wireguard configuration file (for example */etc/wireguard/wgc.conf*) to use PostUp and PosDown.

```
[Interface]
PrivateKey = ...
Address    = ...
Postup     = /etc/wg-client/post-up.sh
PostDown   = /etc/wg-client/post-down.sh
...
```

In addition the wg-client config file, */etc/wg-client/config*, must set the wireguard interface that is used. For example with a line:

```
iface = wg0
```

There helper scripts *post-up.sh* and *post-down.sh* are part of *wg-client*. *wg-client* handles everything else including running *resolv-manager* to monitor */etc/resolv.conf* while the vpn is active to ensure that DNS is resolved using the appropriate DNS servers while wireguard is active as well as restoring the standard *resolv* file when wireguard is shut down.

More information about *resolv-manager* can be found in the man page:

```
man wg-client-resolv-manager
```

2.2 wg-client application

2.2.1 Usage

wg-client is a command line program and can be run in any terminal. It has two primary options, one to start wireguard and one to stop it.

```
wg-client --wg-up  
wg-client --dg-dn
```

It uses a config file (*/etc/wg-client/config*) where the wireguard interface is specified along with optional *ssh* information.

To see the list of available options use *-h*.

The *wg-client* options are described fully in the manual section [Options](#) and the configuration file in section [Configuration](#).

2.3 wg-client-gui application

2.3.1 GUI Usage

The gui app can be run using the *wg-client.desktop* file and can be added to launchers in the usual way. For example in *gnome* simply search applications for *wg-client* and right click to pin the launcher. The gui is built on *PyQt6* which relies on *Qt6*.

The gui has buttons to start and stop wireguard and a button to run *ssh* to set up the listener on the host configured in the config file.

The gui should be left running while the vpn is in use. Pressing quit in the gui will shutdown wireguard and shutdown the *ssh* listener as well.

2.4 Sudoers

To start and stop wireguard *wg-client* uses *wg-quick* (from *wireguard-tools*). Doing so requires root privileges. As a result, any user approved to run wireguard must be granted permission. Any non-root user will need a *NOPASSWD* *sudoers* entry.

You can keep all local sudoers in a single file or in separate files. If in single file, make this one come after any group wheel ones. This is to ensure this one is chosen because sudo uses the last matching entry.

Simply add this sample line adjusting WGUSERS to list whatever user(s) are permitted to run wireguard. If more than one use comma separated list as shown below.

```
User_Alias WGUSERS = alice, bob, sally
WGUSERS ALL = (root) NOPASSWD: /usr/bin/wg-quick
WGUSERS ALL = (root) NOPASSWD: /usr/lib/wg-client/wg-fix-dns
```

If using separate files, then care is needed to ensure this entry comes after any wheel group entries. Where WGUSERS is 1 or more usernames or a group such as *%wgusers*.

Then,

```
visudo /etc/sudoers.d/100-wireguard
```

Edit *WGUSERS* as above.

visudo enforces the correct permissions which should be '0440'. If permissions are too loose, sudo will ignore the file.

Why the prefix number? Because sudo uses the **last** matching entry and we need to be sure the NOPASSWD wg-quick entry comes after any group wheel lines.

For example if there are 2 files in */etc/sudoers.d* - say wg-quick and wheel, where the wheel entry requires a password for members of group wheel.

Now if user listed in wg-quick is also a member of *wheel* group, since wg-quick is first and wheel is second (files are treated in lexical order) the *wheel* one will prevail and user will be prompted for a password when running *sudo /usr/bin/wg-quick*. Not what we want. To fix this use numbers to prefix the sudoers filenames. So in this example it would be:

```
/etc/sudoers.d/010-wheel
/etc/sudoers.d/100-wg-client
```

thereby ensuring that wg-client entries follow the wheel ones.

For convenience this is also noted in the sample file:

```
/etc/wg-client/samples/sudoers.sample
```

```
chmod 440 /etc/sudoers.d/wg-client
```

This can also be achieved with doas or run0.

RECENT CHANGES

Sun Oct 4 06:14:30 AM EDT 2026

- Update Documentation
- Now available on <https://wg-client.readthedocs.io>
- GUI
 - Migrate from PyQt6 to PySide6
 - Fix some GUI related warnings
 - ★ Could not register app ID: Connection already associated with an application ID
 - ★ Trying to disable 0xNNN but 0x0 is focused

CONFIGURATION

wg-client reads its configuration from

```
/etc/wg-client/config
```

Please copy the sample config and edit appropriately. The format is in *TOML* format.

```
# Sample
iface = 'wg0'
ssh_server = 'vpn.example.com'
ssh_pfx = '47'
```

This config file provides:

- **iface** - required
Wireguard interface; defaults to *wg0*. It is *<iface>* of */etc/wireguard/<iface>.conf*
- **ssh_server** - optional
Hostname of the remote ssh server accessible over the vpn; this is where the ssh listening port is run. Hostname must be accessible over the wg vpn.
- **ssh_pfx** - used with **ssh_server**
1 or 2 digit number, 65 or smaller, to be used as ssh listening port number prefix. The port number is of the form *PPxxx*, with *PP* the prefix and *xxx* is taken from the last octet of the wireguard vpn internal IP address.
The prefix can also be given as a range of numbers (*'n-m'*). In this case the prefix used is randomly chosen from that range
Keep in mind that the largest port number is 65535, which limits *ssh_pfx* to be 65 or lower.

The port number chosen will be written to the log file.

The remote ssh host will then listen on *127.0.0.1:<port>*. It will also listen on *<remote-ip-address>:<port>* provided the remote ssh server permits it by having the *sshd* option set:

```
GatewayPorts yes
```


OPTIONS

5.1 wg-client

Summary of available options for wg-client.

Positional argument : Optional

- wireguard interface name
Default interface is taken from *iface* in config file. The config file is chosen by first checking for *./etc/wg-client/config*¹ and then in */etc/wg-client/config*. If not found the wg interface defaults to *wgc*

Options:

These are all the options available in wg-client:

```
positional arguments:
  iface                Optional wg interface (or test-dummy for test mode)

options:
  -h, --help          show this help message and exit
  --test              Test mode
  --wg-up             Turn wireguard on
  --wg-dn             Turn wireguard off
  -- resolv-manager-start
                        Try (re)start resolv-manager daemon - development option
  --ssh-start         Run ssh over vpn to create remote listener
  --ssh-stop          Kill ssh listener if its running
  --ssh-pfx SSH_PFX   ssh port(s). 2 digits: "nn" or "nn-mmm" for range
  --ssh-server SSH_SERVER
                        Remote ssh server to set up listening port
  --show-iface        Report the wg interface name
  --show-ssh-server    Report the ssh server name
  --show-ssh-running   Report if ssh is running
  --show-wg-running    Report if wireguard is running
  --show-resolv-manager
                        Report if resolv-manager is running
  --show-info         Display status - alias for --status
  --status            Display status
  --version           Display version
```

¹ Useful during development and testing

5.1.1 Using ssh

When asking wg-client to start ssh, it runs ssh to the remote host inside the vpn tunnel, sets it to listen on a remote port. Port number used is described in section [Configuration](#).

Note that this blocks waiting for ssh. To stop ssh, simply make a separate invocation of *wg-client -ssh-stop*. If using the GUI tool, simply click the *Stop Ssh* button.

In the event that ssh connection is dropped, it will automatically be restarted. There are normal, quite common situations where ssh process can exit prematurely.

For example:

- After sleep/resume (longer than tcp timeout)
- if remote server sshd restarts (reboot for example)
- changing IP address (e.g. happens when location changes. e.g. Move from hotel to starbucks)

While attempting to reconnect ssh there is a waiting period between each attempt. Currently the wait time is 30 seconds.

The ssh port prefix can be 2 digits: “nn” or a range of 2 digit numbers “nn-mm”. When using a range, prefix used will be randomly drawn from the range. Maximum value is 65. This can also be set in the config file.

LOG FILES

There are 3 kinds of log files: one for resolv-manager, one for command line app and one for the gui app. resolv-manager logs are in /var/log/wg-client and while wg-client and wg-client-gui logs are in:

```
`${HOME}/log/wg-client  
`${HOME}/log/wg-client-gui
```

All log files are rotated with the usual integer suffix scheme.

7.1 Migration from Pre 8.x

Older versions (pre 8.x) need a small change because *resolv-manager* has been re-written in C and now runs a background daemon managing */etc/resolv.conf*.

This is what is needed:

- Create */etc/wg-client/wireguard-resolv.conf* This should be the DNS settings to be used while wireguard is running. Standard *resolv.conf* format (nameserver x.x.x.x)
- confirm that */etc/wg-client/config* exists and sets the wireguard interface. For example:

```
iface = wg0
```

- edit the wireguard client config and update the PostUp and PostDn. For example:

```
[Interface]
PrivateKey = ...
Address = ...
Postup = /etc/wg-client/post-up.sh
PostDown = /etc/wg-client/post-down.sh
```

That should be all that's required.

7.2 Installation

Available on:

- [Github](#)
- [Archlinux AUR](#)

On Arch you can build using the PKGBUILD provided in packaging directory or from the AUR package.

To build manually, clone the repo and do:

```
./scripts/do-build
./scripts/do-install [<destination>]
```

If destination is not specified it defaults to *build/pkg*

7.3 Dependencies

Run Time :

- python (3.13 or later)
- psutil (python-psutil)
- dateutil
- libcap
- pynotify (python-notify)
- openssl
- pyconcurrent
- PySide6 / Qt6 (for gui)
- hicolor-icon-theme
- bash
- glibc

Building Package:

- git
- meson
- meson-python
- rsync

7.4 Log files

Each application has it's own log file. They are located in the home directory :

```
${HOME}/log/wg-client  
${HOME}/log/wg-client-gui
```

Each log file is rotated with using numerical suffix.

7.5 License

Created by Gene C. and licensed under the terms of the GPL-2.0-or-later license.

- SPDX-License-Identifier: GPL-2.0-or-later
- SPDX-FileCopyrightText: © 2023-present Gene C <arch@sapience.com>

8.1 resolv-manager

8.1.1 Manages /etc/resolv.conf while using Wireguard

SYNOPSIS

/usr/lib/wg-client/resolv-manager -i <iface> -t <task-name> [*options* ...]

DESCRIPTION

Part of the wg-client package. When wireguard VPN is running DNS is typically used via the VPN tunnel. When wireguard is started an appropriate /etc/resolv.conf should be installed and when it is stopped the standard version should be restored.

This is simple to do using PostUp and PostDown in the wireguard config file. For example if the config file is /etc/wireguard/wgc.conf, so the wireguard interface is wgc then it would have:

```
[interface]
...
PostUp    = /etc/wg-client/post-up.sh wgc /etc/wg-client/wireguard-resolv.conf
PostDown  = /etc/wg-client/post-down.sh
```

Where the file /etc/wg-client/wireguard-resolv.conf is the resolv.conf to use while the VPN is running.

With no arguments post-up.sh reads the wireguard interface from /etc/wg-client/config and uses /etc/wg-client/wireguard-resolv.conf for the VPN resolv.conf.

post-up.sh uses resolv-manager to save a copy of the current standard resolv.conf and install the wireguard provided file to /etc/resolv.conf by running:

```
resolv-manager -i wgc -t wireguard -f /etc/wg-client/wireguard-resolv.conf -l /var/log/wg-client
```

The standard resolv.conf is saved to resolv.conf.saved and the caller provided wireguard file is tagged and saved to resolv.conf.wg.

When shutting down wireguard, post-down.sh uses it to restore the standard resolv.conf by using:

```
resolv-manager -t standard -l /var/log/wg-client
```

In this case the standard resolv.conf is restored from the saved version.

The wireguard version has a comment added as the first line in the file in order to mark it as to be used only while wireguard is active.

With the above DNS will be work correctly when starting and stopping wireguard using:

```
wg-quick up wgc
```

and

```
wg-quick down wgc
```

It is quite normal that `/etc/resolv.conf` can be changed by some network related events. For example, This can happen after a sleep/resume cycle, when a DHCP lease renews or when a new WiFi network is connected.

While wireguard itself is largely immune to such changes, since it uses UDP, DNS needs to be fixed up to keep using the wireguard version of `resolv.conf`.

`resolv-manager` handles this. When **wg-client -wg-up** is used to start wireguard (or the GUI which runs the same command) it starts *ressolve-manager* in *monitor* mode.

MONITOR MODE

wg-client -wg-up starts wireguard and then runs:

```
resolv-manager -i wgc -t monitor
```

This runs as a background daemon and monitors `/etc/resolv.conf` for any changes. This is done using *inotify* whereby it asks the kernel to wake it up when the file changes. This is extremely efficient and allows it to sleep until the kernel wakes it up.

When it detects that `resolv.conf` has changed, it saves the new standard version and re-installs the VPN `resolv.conf` so that DNS continues to use the tunnel.

It also monitors `/sys/class/net/wgc` (or whatever interface is being used) and when it detects that wireguard is no longer running it shuts itself down.

It can also monitor has an option to monitor a **semaphore** file and deleting that semaphore file, signals the monitor to shutdown. `wg-client` uses a semaphore file.

The reason is because `resolv-manager` uses locking to guarantee there is only one version running at a time. The semaphore gives **wg-client** the ability to instruct `resolv-manager` to shut down before it turns off wireguard. This way wireguard's `PostDown` can safely use `resolv-manager` to restore the standard `resolv.conf` since the monitor instance is no longer running.

Clearly there cannot be more than one program monitoring and modifying the same files. File Locking is a sound way to make sure this does not happen. The lock file used is **`/run/lock/wg-client/lock`**. `resolv-manager` will only monitor or change any files after it has acquired the lock.

SECURITY

Since resolv-manager needs privileges to modify file in /etc and it is run by non-root user, it uses just enough **capabilities** to do its job.

These are all that are required:

```
/usr/bin/setcap cap_chown,cap_dac_override=ep /usr/lib/wg-client/resolv-manager
```

The Arch PKGBUILD sets these capabilities when the package is installed.

It is designed to be run as ordinary unprivileged user.

If it is run as root, then it drops privileges to nobody and retains the capabilities to function properly. Starting wireguard itself needs root privileges so resolv-manager is started by root in the PostUp and PostDown settings.

OPTIONS

The required options are **-iface** and **-task**.

```
-t, --task      <name>    Task name is one of: s[tandard], w[ireguard] or m[onitor]
-i, --iface    <iface>    The wireguard interface name (e.g. wg0)
-l, --logdir   <dir>      Log directory
-n, --numlogs  <num>      Number of log files to keep in the rotation.
-d, --daemon   <num>      Run as a background daemon.
-f, --file     <file>      Used with -t wireguard to provide the wireguard resolv.conf to use
-s, --semaphores <file>    Watch this file and exit if removed or not found (monitor mode)
-T, --testing   <file>      Developer option. Flies are relative to the current directory.
                        For example it looks for ./etc/resolv.conf
--help, -h      <file>      Show help message and exit.
```

At least one task is required and both the *wireguard* and *monitor* tasks both need the interface to be provided as well.

ERRORS

The exit codes are taken from “sysexits.h”:

- EX_OK
A successful termination.
- EX_USAGE
Problematic command line arguments.
- EX_OSFILE
File related errors, typically unable to open a file.
- EX_TEMPFAIL
Not ready as wireguard is not running.
- EX_OSERR
Likely due to failure to daemonize
- EX_IOERR
IO errors are likely to to reading or writing files. Probably /etc/resolv.conf et al.
- EX_SOFTWARE
Some other problem occurred. Most likely the monitor encountered an unrecoverable error related to inotify.

EXAMPLES

Run as a daemon to monitor the dns resolv.conf file for changes:

```
/usr/lib/wg-client/resolv-monitor -i wg0 -t monitor -d -l /var/log/wg-client
```

Run as non-privileged user and exit when a semaphore file is removed:

```
/usr/lib/wg-client/resolv-monitor -i wg0 -t monitor -s /tmp/i-am-running
```

Install a wireguard file to */etc/resolv.conf* and save current standard file:

```
/usr/lib/wg-client/resolv-monitor -i wg0 -t wireguard -f /etc/wg-client/wg-resolv.conf
```

Restore the standard */etc/resolv.conf* back to */etc/resolv.conf*:

```
/usr/lib/wg-client/resolv-monitor -t standard -l /var/log/wg-client
```

SEE ALSO

wg(8), wg-quick(8)